Artifact Appendix: Vista: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing

Submission ID #217

1 Badge Target

We target all three artifact badges: Available, Functional, and Results Reproduced.

- Repository: https://github.com/benyucong/rl-quantum
- Archive: https://doi.org/10.5281/zenodo.19712131

2 Hardware and Software Requirements

Test mode	Requirement
CPU-only inspection	Python 3.10 or 3.11. No GPU is required for the quickest check, extracted Tables, or packaged-figure redraws.
Evaluation stack	Install the packages in `quantum-code-generation/code/evaluation/requirements.txt`, including Qiskit 1.2.4, qiskit-aer 0.16.1, and qiskit-qasm3-import 0.5.1.
Generation	Regenerating model outputs requires a GPU-capable machine with enough memory for the selected 3B/4B checkpoint.
Industrial LLM API baselines	Table 1 API baselines used DeepSeek-V3, GPT-5, and GPT-4o. Rerunning these baselines requires provider API credentials, quota, and network access; inspecting archived raw JSONs does not.
Optional live QPU rerun	Figures 10–14 can be inspected from archived hardware logs. A live rerun additionally requires provider credentials and API access for IBM or IQM cloud quantum computers, subject to queue and quota availability.
Training / tested platform	Tested on the Aalto Triton HPC system, the LUMI HPC system, and MareNostrum5 hosted by BSC, Spain, in Linux + Slurm workflows with module-based environments such as `scicomp-python-env` and `scicomp-llm-env`. Paper-scale Vista training used 8 AMD MI250X GPUs or 8 NVIDIA H100 GPUs. Smaller single-node Slurm launchers are also included for sanity checks.

3 Packaged Data Replotting Check

Quick check: CPU-only table and figure reproduction. This independent CPU-only check displays the packaged Tables and replots the packaged figure data; it does not regenerate model outputs or retrain Vista. Figures 10–14 are regenerated in this mode from archived hardware/QPU report JSONs, so no live cloud-QPU access is required. Expected hardware: any CPU-only machine.

• **Step 1.** Clone and enter the repository.

```
git clone https://github.com/benyucong/rl-quantum.git
cd rl-quantum
```

• **Step 2.** Summarize the packaged evaluation outputs and show the extracted Tables 1–3.

```
python3 artifact/scripts/summarize_eval_outputs.py \
  quantum-code-generation/code/evaluation/out
python3 artifact/scripts/show_paper_tables.py all
```

• **Step 3.** Create a plotting environment, install the lightweight figure dependencies, and redraw the packaged figures from `vista_draw`.

```
python3 -m venv .venv-fig
source .venv-fig/bin/activate
pip install -r artifact/requirements-figures.txt
python3 artifact/scripts/draw_vista_figures.py \
  --input-dir vista_draw \
  --output-dir artifact_runs/paper_figures \
  --strict
```

Table 1 maps Figures to the corresponding artifact tasks. The complete script/input map is in `artifact/tables/figure_artifact_map.csv`.

Table 1: Figure-to-artifact task correspondence.

Item	Artifact task	Main evidence
Fig. 1	`design_trace`	Training, verifier, and reward code inspection
Fig. 2	`per_primitive`	`per_primitive_hqcr.csv`
Fig. 3	`relative_entropy`, `box`	`relative_entropy_table.csv`, `box.csv`
Fig. 4	`scalability_qubits`	`scalability_table_0.3.csv`
Fig. 5	`scalability_gates_depth`	Gates/depth scalability CSVs
Fig. 6	`training_dynamics`	Training-dynamics CSVs
Fig. 7	`verifier_efficiency`	Verifier-efficiency CSV
Fig. 8	`training_logs`	Budget-matched outputs from `logs.csv`
Fig. 9	`stage_cost`	Verifier timing from `logs.csv`
Fig. 10	`helmi_reward_stability`, `real_device_tradeoff`	Archived hardware/QPU report JSON
Fig. 11	`latency_breakdown`	Archived hardware/QPU report JSON
Fig. 12	`helmi_reward_stability`	Archived hardware/QPU report JSON
Fig. 13	`training_logs`	`budget_matched_q5_*` outputs
Fig. 14	`real_device_tradeoff`	Hardware quality–efficiency table/report
Fig. 15	`dataset_trace`	Graph-generation pickles and Table 3

4 Step by Step Reproduction Instructions

Reproduction 1: Checkpoint-based. This reproduction path uses the released checkpoint to generate new outputs, evaluate them, and redraw the evaluator-backed plots. Expected hardware: one GPU machine with enough memory for the released 4B checkpoint.

• **Step 1.** Clone and enter the repository.

```
git clone https://github.com/benyucong/rl-quantum.git
cd rl-quantum
```

• **Step 2.** Create an environment and install the generation, evaluation, and figure dependencies.

```
python3 -m venv .venv-ae
source .venv-ae/bin/activate
pip install -r artifact/requirements-figures.txt
cd quantum-code-generation/code/generation
pip install -r requirements.txt
cd ../evaluation
pip install -r requirements.txt
cd ../../..
```

• **Step 3.** Generate checkpoint outputs, evaluate them, and redraw the evaluator-backed plots.

```
bash artifact/scripts/run_quantum_experiment.sh \
  --n-samples 50 \
```

```
  --out-dir artifact_runs/reviewer_smoke
EVAL_DIR=artifact_runs/reviewer_smoke/evaluation
FIG_DIR=artifact_runs/reviewer_smoke/figures
python3 artifact/scripts/draw_vista_figures.py \
  --evaluation-dir "$EVAL_DIR" \
  --only box,relative_entropy \
  --output-dir "$FIG_DIR" \
  --strict
```

Reproduction 2: Retrain-based. This reproduction path launches GRPO/FSDP retraining from the training scripts and is intended only for evaluators with access to large-scale GPU resources. Expected hardware: paper-scale retraining used 8 AMD MI250X GPUs or 8 NVIDIA H100 GPUs.

• **Step 1.** Clone and enter the repository.

```
git clone https://github.com/benyucong/rl-quantum.git
cd rl-quantum
```

• **Step 2.** Activate the training environment and set the Hugging Face cache path.

```
source .venv/bin/activate
export HF_HOME=$(pwd)/data/huggingface
export VISTA_REWARD_ABLATION=full
# Table 2:
# full, no_ev, no_re, no_opt, validity_only
```

• **Step 3.** Launch retraining directly or submit a cluster wrapper.

```
bash examples/train/quantum/train_qwen_4B_quantum.sh
# or use a cluster launcher:
# sbatch examples/train/quantum/train_hpc.sh
# sbatch examples/train/quantum/train_mn5.sh
```

5 Evaluation Claims and Reproduction Plan

Claim	Result and action
End-to-end quality	**Reported result:** Reported Table 1 gives Vista Pass@1 SCR 99.31%, SREV 22.41%, RE 11.61, HQCR 17.24%; Pass@10 SCR 100%, SREV 33.10%, RE 8.48, HQCR 27.24%. **Reviewer action:** Recompute `summary_stats_*.json` from raw generation JSONs with `evaluate_samples.py`.
Reward ablation	**Reported result:** Reported Table 2 gives full Vista as best overall and removing RE as the largest drop. **Reviewer action:** Train/generate each variant by setting `VISTA_REWARD_ABLATION` to `full`, `no_ev`, `no_re`, `no_opt`, or `validity_only`, then evaluate the raw outputs.
Verifier efficiency	**Reported result:** Figures 7–9 report verifier time drops from 8.20s to 4.63s per iteration, a 1.77x speedup. **Reviewer action:** Inspect timing logs or rerun gated/non-gated variants under the same budget.
Scaling and robustness	**Reported result:** Figures 2, 4, and 5 report stronger results across thresholds, graph primitives, and 8–16 qubit sizes. **Reviewer action:** Redraw with `draw_vista_figures.py`.
Hardware deployment	**Reported result:** Figures 10–14 report preserved objective quality with shorter scheduled circuit duration on real/emulated QPU backends. **Reviewer action:** Inspect archived hardware logs; a live rerun additionally needs IBM/IQM cloud-QPU API credentials and queue access.

6 Artifact Overview

This artifact supports the CAIS 2026 paper "Vista: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing." The artifact contains the training code for Vista, the staged OpenQASM verifier used during agentic reinforcement learning, generation scripts for Vista and baseline checkpoints, and offline evaluation scripts for reproducing the reported semantic metrics.

The main artifact components are grouped by role:

- **Training and RL orchestration.** `examples/train/quantum/` contains the GRPO/FSDP training scripts for the 4B quantum circuit model.
- **Verifier and reward implementation.** `verl_tool/servers/tools/quantum_cpu.py` provides the quantum verifier service used during tool-augmented rollouts, and `verl_tool/servers/tools/utils/quantum_reward_cal.py` implements staged rewards for syntax feasibility, behavior similarity, expectation-value objective quality, and utility-oriented optimization.
- **Model-output generation.** `quantum-code-generation/code/generation/` contains scripts for generating OpenQASM circuits from Vista and baseline checkpoints.
- **Offline evaluation and metric recomputation.** `quantum-code-generation/code/evaluation/` parses generated QASM, simulates circuits, computes relative entropy, checks expectation values, and writes summary statistics; `artifact/scripts/run_quantum_experiment.sh` is the reviewer wrapper for generation plus evaluation or evaluation of an existing raw generation file.
- **Dataset construction and packaged inputs.** `quantum-code-generation/code/data_generation/src/algorithms/` contains graph-generation source files and included pickle inputs for the supported graph optimization primitives.
- **Figure and table reproduction.** `artifact/scripts/draw_vista_figures.py` and `vista_draw/` provide headless wrappers and plot-ready inputs for regenerating Figures; `artifact/tables/` and `artifact/scripts/show_paper_tables.py` provide extracted CSV values for Tables 1–3 and the figure/table map.
- **Released checkpoints and public data.** The artifact references the Hugging Face RL checkpoint `Benyucong/rl_quantum_4b`, SFT initialization checkpoint `Benyucong/sft_quantum_circuit_gen_4B`, and dataset `Benyucong/graph-data-quantum-rl`.